openCV with Python

David Hsu

Installations - windows

- Python 2.7 or 3.5
  https://www.python.org/downloads/
- Frequently used Libraries : Numpy, Matplotlib, Scipy
  http://www.scipy.org/
- Or try Anaconda (Python + popular libraries)
  https://www.continuum.io/downloads
- openCV 3.0.0
  http://opencv.org/downloads.html
- Copy "cv2.pyd" from path\opencv\build\python\2.7\x86(x64)
  Paste "cv2.pyd" to path\Anaconda\Lib\site-packages
  In python shell : import cv2
  cv2.__version__ (two underscores)

'3.0.0'

Installations - mac

- Python 2.7 or 3.5 or Anaconda
  https://www.python.org/downloads/
  https://www.continuum.io/downloads
- Frequently used Libraries : Numpy, Matplotlib, Scipy
  in Terminals :
  pip install –U numpy scipy matplotlib (for Python 2.X)
  pip3 install –U numpy scipy matplotlib (for Python 3.X)
- Download openCV http://opencv.org/downloads.html
- Install Xcode https://developer.apple.com/xcode/
- Install Cmake https://cmake.org/download/
- Build openCV for Python
  http://luigolas.com/blog/2014/09/15/install-opencv3-with-python-3-mac-osx/

Ex1: read/show images

- Put the image (lena.bmp) and cv_ex1.py in the same folder
- Python 3.5.0 (v3.5.0:374f501f4567, Sep 12 2015, 11:00:19)
  [GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin

```
========== RESTART: /Users/david/Desktop/PythonXopenCV/cv_ex1.py ==========
(512, 512, 3)
pressed key :  -1
```

Ex1: read/show images

```
import cv2

# read the color image
img1 = cv2.imread('lena.bmp')
# img.shape : size of the image --> row,column,depth
print(img1.shape)
# pop up window for image show
# cv2.imshow(title_name, image_array)
cv2.imshow('image color',img1)

# read the color image and turn it to grayscale
# cv2.imread(image_name,flag)
# flag=1 (defalut): color , flag=0:grayscale
img2 = cv2.imread('lena.bmp',0)

# system will pause until it gets pressed key in 5000 ms
p=cv2.waitKey(5000)
# show the pressed key in ASCII code, if no key is pressed, p=-1
print('pressed key : ', p)

cv2.imshow('image gray',img2)

# cv2.waitKey(0) will pause the system until any key is pressed
cv2.waitKey(0)
# assign the Window to be closed
cv2.destroyWindow('image gray')

cv2.waitKey(0)
# close all windows
cv2.destroyAllWindows()
```

Ex2: open videos

- Put the video (test.mp4) and cv_ex2.py in the same folder
-

Ex2: open videos

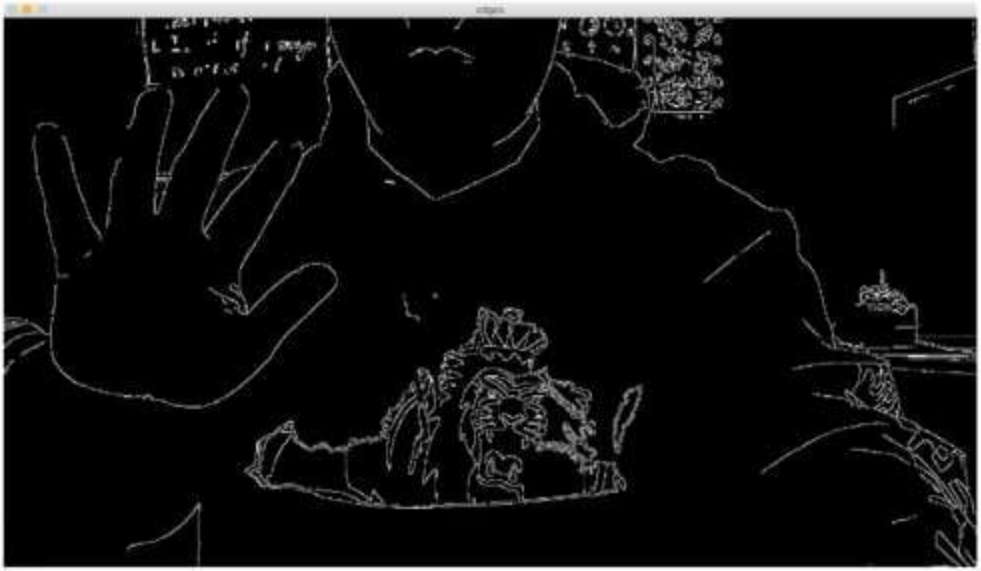
```
import cv2

# open the video
cap = cv2.VideoCapture("test.mp4")

if(cap.isOpened()):
    print("Video opened")
    # set the frame speed in cv2.waitKey
    # set the key 'q' to quit showing frames if it is pressed
    while(cv2.waitKey(30) != ord('q')):
        ret, frame = cap.read()
        # ret is True : frame got
        if ret == False:
            print("Video ends")
            break

        cv2.imshow("Video",frame)

else:
    print("Opening video failed")

cap.release()
cv2.destroyAllWindows()
```

Ex3: using webCam

- Very similar to video open, just change the argument to be 0
-

```
import cv2

# set the argument to be 0
# the index number is to specify which camera will be used
cap = cv2.VideoCapture(0)

if(cap.isOpened()):
    print('FaceTime')
    # press q to quit the webCam
    while(cv2.waitKey(3) != ord('q')):
        ret, frame = cap.read()
        cv2.imshow('webCam',frame)

else:
    print('webCam is failed to use')

cap.release()
cv2.destroyAllWindows()
```

Ex4: edges detection

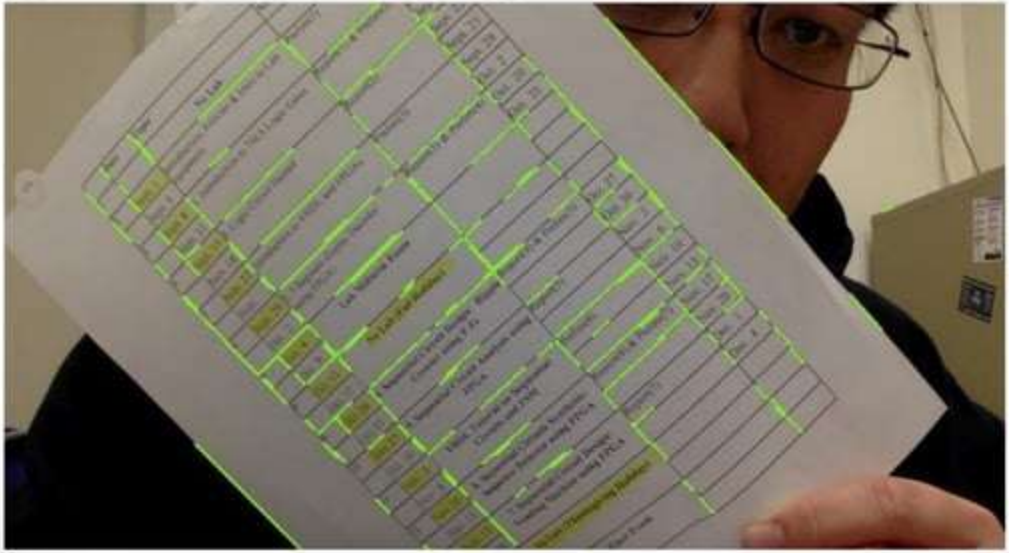
- Adopt “canny” method to achieve edges detection
- 

Ex4: edges detection

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)

if(cap.isOpened()):
    print("webCam opened")
    while(cv2.waitKey(3) != ord('q')):
        ret, frame = cap.read()

        # Canny method for edges detection
        # argument: image_array, minVal, maxVal
        # above maxVal: sure-edges, below minVal : non-edges
        # between maxVal and minVal : depend on the connectivity to sure-edges
        edges = cv2.Canny(frame,100,200)

        cv2.imshow('edges',edges)

else:
    print("Opening webCam failed")

cap.release()
cv2.destroyAllWindows()
```

Ex5: lines detection

- Lines detection using Hough Transform
  → line equation using (r, thita)



Ex5: lines detection

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)
if(cap.isOpened()):
    print("webCam opened")
    while(cv2.waitKey(3) != ord('q')):
        ret, frame = cap.read()
        # Canny method for edges detection
        edges = cv2.Canny(frame,100,200)
        # Hough transform for finding lines
        # argument : edges_array,
        #            scaling for r in Hough Transform : 1 pixel
        #            scaling for thita in Hough Transform : 1 degree (3.14/180 in rad)
        #            threshold to define lines
        #            min line length (optional)
        #            max line gap (optional)
        lines = cv2.HoughLinesP(edges,1,np.pi/180,200,100,10)
        # drawing out the detected lines in green (0,255,0)
        # argument : (draw on)image_array, (from)point1, (to)point2
        for x1,y1,x2,y2 in lines[:,0,:] :
            cv2.line(frame,(x1,y1),(x2,y2),(0,255,0),2)

        cv2.imshow("webCam",frame)
else:
    print("Opening webCam failed")

cap.release()
cv2.destroyAllWindows()
```

Ex6: object tracking using color filtering

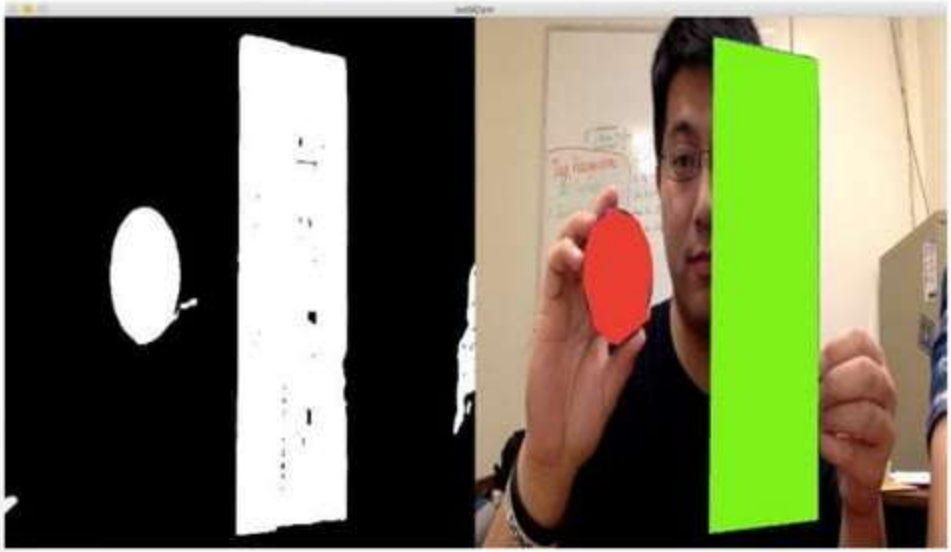
- using **color** as features : quick and dirty
- Detecting something in blue

Ex6: object tracking using color filtering

```
import cv2
import numpy as np
cap = cv2.VideoCapture(0)
if(cap.isOpened()):
    print("webCam opened")
    while(cv2.waitKey(3) != ord('q')):
        ret, frame = cap.read()
        # convert color space from BGR to HSV
        hsv_image = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
        # set up lower and upper bound for color range of blue
        lowBound = np.array([110,50,50])
        upBound = np.array([130,255,255])
        # filtering by color
        mask = cv2.inRange(hsv_image,lowBound,upBound)
        # image dilation
        kernel = np.ones((5,5),np.uint8)
        mask = cv2.dilate(mask,kernel,iterations = 1)
        # combined two images (filter_image,original_image) in one window
        h1,w1 = mask.shape
        h2,w2,d2 = frame.shape
        imgCom = np.zeros((max([h1,h2]),w1+w2,3), dtype='uint8')
        imgCom[:h1,:w1,0] = mask
        imgCom[:h1,:w1,1] = mask
        imgCom[:h1,:w1,2] = mask
        imgCom[:h2,w1:w1+w2,:] = np.dstack([frame])
        # image resize
        imgCom = cv2.resize(imgCom,(w1,h1),interpolation=cv2.INTER_LINEAR)
        cv2.imshow("webCam",imgCom)
cap.release()
cv2.destroyAllWindows()
```

Ex7: contours classification

- blue objects detextion → shapes recognition
- Rectangle → shown in green
- multiple-sides-polygon (like circles) → shown in red

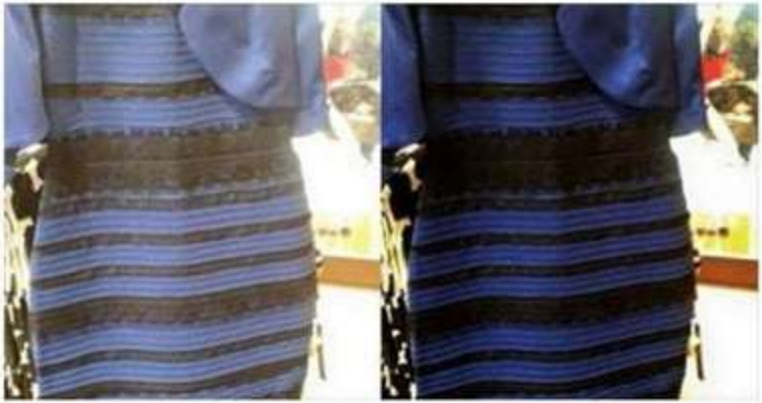


Ex7: contours classification

```
if(cap.isOpened()):
    print("webCam opened")
    while(cv2.waitKey(3) != ord('q')):
        ret, frame = cap.read()
        # convert color space from BGR to HSV
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
        # set up lower and upper bound for color range of blue
        lowBound = np.array([110,50,50])
        upBound = np.array([130,255,255])
        # filtering by color
        mask = cv2.inRange(hsv,lowBound,upBound)
        # image dilation
        kernel = np.ones((5,5),np.uint8)
        mask = cv2.dilate(mask,kernel,iterations = 1)
        # contour extraction
        img, cnts, flag = cv2.findContours(mask.copy(), cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
        # show 2 objects (or more, depend on your setting)
        cnts = sorted(cnts, key = cv2.contourArea, reverse = True)[:2] # 2
        for c in cnts:
            peri = cv2.arcLength(c,True)
            approx = cv2.approxPolyDP(c, 0.01*peri, True)
            if (len(approx)==4): # if it is 4-sides polygon
                cv2.drawContours(frame,[approx],0,(0,255,0),-1)
            if (len(approx)>10): # if the polygon is more than 10 sides (may be circles)
                cv2.drawContours(frame,[approx],0,(0,0,255),-1)

        # combined two images (filter_image,original_image) in one window
        h1,w1 = mask.shape
        h2,w2,d2 = frame.shape
        imgCom = np.zeros((max([h1,h2]),w1+w2,3), dtype='uint8')
```

Lightening condition invariant?

- Color looks different with lightening issue.

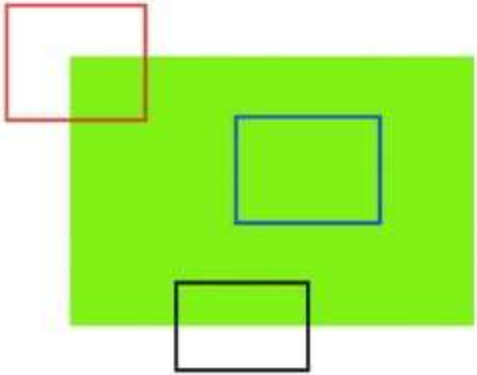


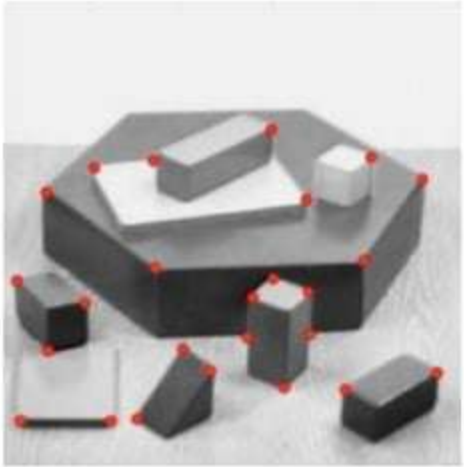
Why that dress looks white and gold: It's overexposed

If you're seeing white and gold, you can correct the photo for overexposure... and it magically turns blue and black.

Finding corners' features

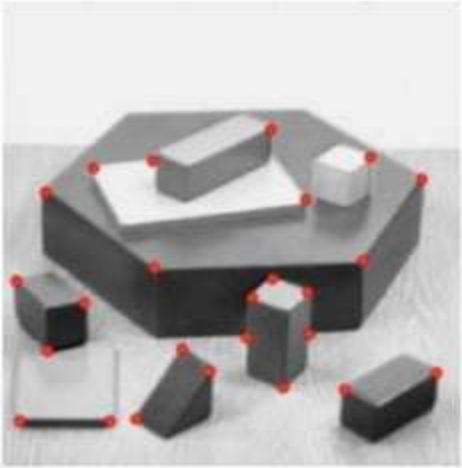
- Color invariant, Scale invariant

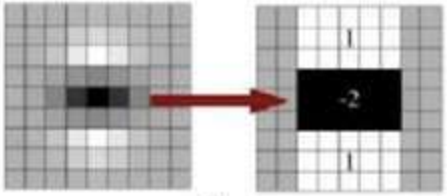




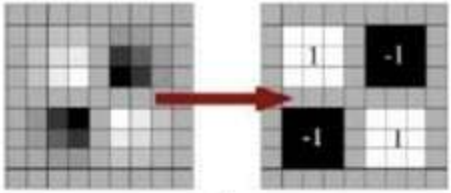
Finding corners' features

- Color invariant, Scale invariant





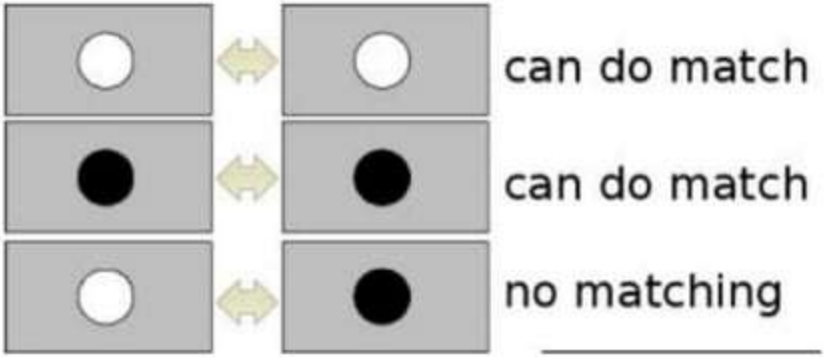
L_{yy}



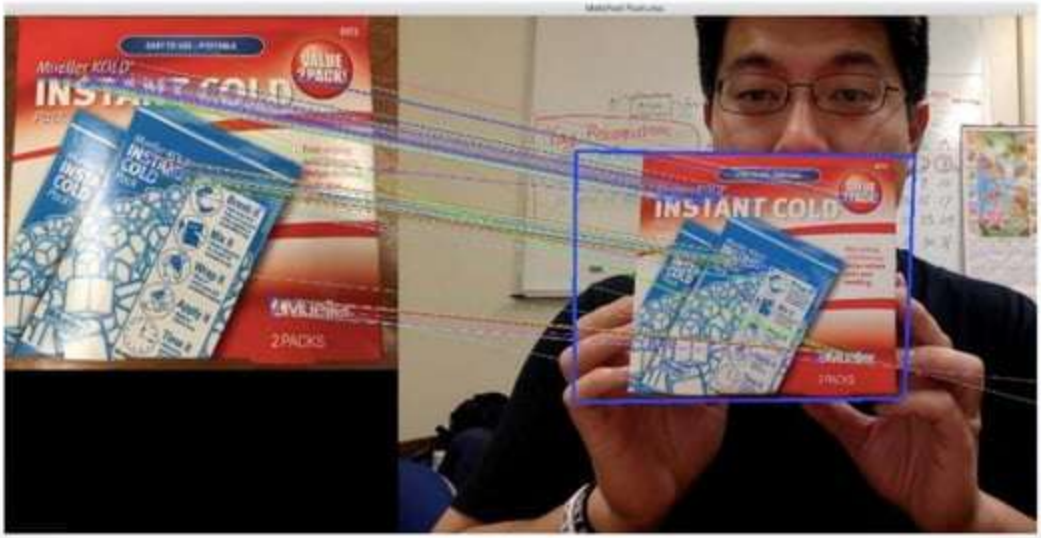
L_{xy}

Features matching

- features matching with two images



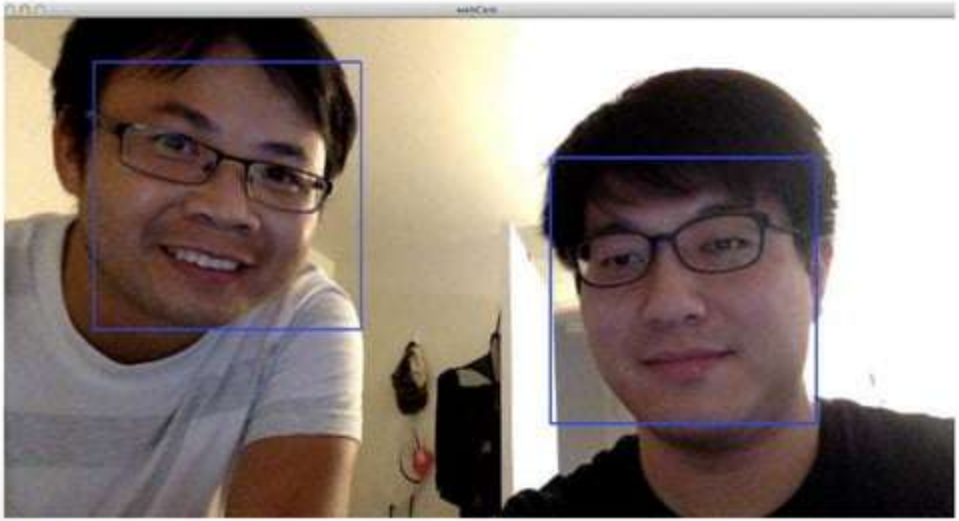
Ex8: object tracking using corners' features

- Features method : ORB (Oriented FAST and Rotated BRIEF)
- Matching method : BruteForceMatcher
- 

Ex8: object tracking using corners' features

```
drawMatchingPoints = True # show Matching Points or not
threshold = 0.08 # threshold for detection
numPoints = 100 # number of points selected (best matching)

img1 = cv2.imread('target.bmp') # target image
h1,w1,d1 = img1.shape
corners1 = np.float32([ [0,0],[0,h1-1],[w1-1,h1-1],[w1-1,0] ]).reshape(-1,1,2)
# detector
orb = cv2.ORB_create()
kp1, des1 = orb.detectAndCompute(img1,None)

cap = cv2.VideoCapture(0); # access webCam
while(True):
    ret, frame = cap.read()
    #img2 = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    img2 = frame # scene from webCam
    h2,w2,d2 = img2.shape

    # find keypoints and descriptors
    kp2, des2 = orb.detectAndCompute(img2,None)

    # match
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck = True)
    matches = bf.match(des1,des2)
    #print(len(matches))

    matches = sorted(matches, key = lambda x:x.distance)
    good = matches[:numPoints] # select the best "numPOints" points
```

Ex9: Face recognition/tracking

- Download the training model (haarcascade_frontalface_default.xml') and put it in the same folder with the code cv_ex9.py.
- 

Ex9: Face recognition/tracking

```
import cv2

cap = cv2.VideoCapture(0)

if(cap.isOpened()):
    print("webCam opened")
    # training model for face and eyes
    face = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
    eyes = cv2.CascadeClassifier('haarcascade_eye.xml')

    while(cv2.waitKey(3) != ord('q')):
        ret, frame = cap.read()
        # convert to grayscale image
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        # face detection
        faceDetect = face.detectMultiScale(gray, 1.3, 5)
        # draw a bounding box for the detected face
        for (x,y,w,h) in faceDetect:
            frame = cv2.rectangle(frame,(x,y),(x+w,y+h),(255,0,0),2)

        cv2.imshow("webCam", frame)

else:
    print("Opening video failed")

cap.release()
cv2.destroyAllWindows()
```

References

- You can access the example from

https://github.com/ntudavid/CV_lab

- Reference : OpenCV – Python Tutorials

https://opencv-python-tutroals.readthedocs.org/en/latest/